

E-COMMERCE · ANALYTICS · QUERY TUNING

PostgreSQL 종합실습 4

SQL 분석 및 성능 개선 보고서

전자상거래 주문·상품·고객·재고·리뷰 데이터를 대상으로 매출, RFM, 재구매율, 재고 위험, 쿠폰 효과를 분석하고 실행 계획과 인덱스를 이용해 성능을 비교했다.

소속

광주 3반

이름

신형섭

작성일

2026-07-24

DBMS

PostgreSQL 17

분석 도구

DBeaver · VS Code

문항

Q1 ~ Q11

비즈니스 분석

GMV · AOV · RFM · 재구매율 · Top-N

성능 진단

EXPLAIN ANALYZE · BUFFERS · 실제 행 수

성능 개선

부분·커버링 인덱스 · Window · Materialized View

제출물 구성 — Q1~Q10의 튜닝 전·후 SQL 실행 화면과 JSON 실행 계획, Q11 안전 나눗셈 함수, Materialized View 결과를 문항 순서대로 정리했다.

LAB OVERVIEW

분석 범위와 튜닝 절차

02

전자상거래 데이터 영역

- orders · order_items — 주문 상태, 채널, 쿠폰, 판매 금액
- products · categories — 제품 및 카테고리 성과
- customers — RFM, 재구매율, 상위 고객
- inventory · reviews — 품질 위험 및 효자상품

성능 검증 기준

- 동일 결과를 반환하는 튜닝 전·후 SQL 비교
- EXPLAIN (ANALYZE, BUFFERS) JSON 원본 보존
- Execution Time, Scan, Join, Buffers 변화 확인
- 인덱스가 선택되지 않은 경우 데이터 규모와 선택도 분석

튜닝 전
실행계획부분·커버링
인덱스SQL 구조
단순화튜닝 후
실행계획MV
리포트측정 문항
10개측정 합계 · 전
87.840 ms측정 합계 · 후
65.356 ms전체 감소율
25.6%

결과 동일성

튜닝 전·후 캡처에서 동일한 컬럼과 결과가 반환되는지 확인했다.

실행 계획 원본

문항별 JSON 파일에 SQL 전체와 실행 계획, Planning/Execution Time을 보존했다.

정직한 평가

Q1·Q5·Q8처럼 개선되지 않은 측정도 포함하고 원인을 기록했다.

측정 주의

밀리초 단위 단일 실행은 캐시와 시스템 부하의 영향을 받을 수 있다.

ROLLING 1 MONTH · GMV

03

지난 한 달 실제 판매 금액

튜닝 전

3.484 ms

튜닝 후

3.502 ms

변화

0.5% 증가

orders 접근

Index Only

튜닝 전

Seq Scan

```

-- =====
-- Q1. 지난 한 달간 실제 팔린 총 금액 보기
--   - 주문 상태: paid, shipped, delivered
--   [제출] 튜닝 전 쿼리 및 실행 결과 화면
-- =====
SELECT
  COALESCE(SUM(oi.line_total), 0) AS total_sales_amount
FROM ecom.orders AS o
JOIN ecom.order_items AS oi
  ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
  AND o.order_ts >= CURRENT_TIMESTAMP - INTERVAL '1 month';

```

orders(+) 1 X

SELECT COALESCE(SUM(oi.line_total), 0) AS total_sales_amount

total_sales_amount
874,043.78

결과 874,043.78 — orders와 order_items를 Hash Join하고 orders를 순차 탐색했다.

튜닝 전 JSON

튜닝 후

Index Only Scan

```

-- =====
-- Q1. 튜닝 후 쿼리
--   [제출] 튜닝 후 쿼리 및 실행 결과 화면
--   [비교] EXPLAIN ANALYZE로 튜닝 전후 실행 계획과 실행 시간 비교
--   [튜닝] 판매 상태+기간 부분 인덱스와 order_items 커버링 인덱스 활용
-- =====
EXPLAIN (ANALYZE, BUFFERS)
SELECT
  COALESCE(SUM(oi.line_total), 0) AS total_sales_amount
FROM ecom.orders AS o
JOIN ecom.order_items AS oi
  ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
  AND o.order_ts >= CURRENT_TIMESTAMP - INTERVAL '1 month';

```

orders(+) 1 X

SELECT COALESCE(SUM(oi.line_total), 0) AS total_sales_amount FROM e

total_sales_amount
872,053.12

부분 인덱스 idx_orders_sales_ts_customer를 사용해 orders 버퍼가 줄었으나 order_items 전체 스캔 비용이 남았다.

튜닝 후 JSON

판단 — 최상위 공유 버퍼는 177에서 151로 감소했지만 실행 시간은 0.018ms 증가해 사실상 동일했다. 선택된 주문이 많고 order_items 11,012행을 모두 읽으므로 인덱스 전환만으로는 유의미한 개선이 없었다.

MONTHLY KPI · AOV

월별 주문 수·매출·평균 주문액

04

튜닝 전
12.356 ms튜닝 후
10.908 ms감소율
11.7%결과 그룹
6개월

튜닝 전

2단계 HashAggregate

```

WITH order_totals AS (
  SELECT
    o.order_id,
    date_trunc('month', o.order_ts) AS order_month,
    SUM(oi.line_total) AS order_amount
  FROM ecom.orders AS o
  JOIN ecom.order_items AS oi
    ON oi.order_id = o.order_id
  WHERE o.order_status IN ('paid', 'shipped', 'delivered')
  GROUP BY
    o.order_id,
    DATE_TRUNC('month', o.order_ts)
)
SELECT
  order_month,
  COUNT(*) AS order_count,
  SUM(order_amount) AS total_sales_amount,
  AVG(order_amount) AS avg_order_amount
FROM order_totals
GROUP BY order_month
ORDER BY order_month;

```

order_month	order_count	total_sales_amount	avg_order_amount
2026-03-01 00:00:00.000 +0900	207	123,172.21	595.0348309179
2026-04-01 00:00:00.000 +0900	1,189	762,757.73	641.5119680404
2026-05-01 00:00:00.000 +0900	1,250	862,701.14	690.160912
2026-06-01 00:00:00.000 +0900	1,373	909,984.31	662.7708011653
2026-07-01 00:00:00.000 +0900	1,044	706,399.18	676.6275670498
2026-08-01 00:00:00.000 +0900	1	177.63	177.63

주문별 금액을 먼저 계산한 뒤 월별로 다시 집계해 상품 행 수가 주문 수를 부풀리지 않도록 했다.

튜닝 전 JSON

튜닝 후

통계 갱신

```

EXPLAIN (ANALYZE, BUFFERS)
WITH order_totals AS (
  SELECT
    o.order_id,
    DATE_TRUNC('month', o.order_ts) AS order_month,
    SUM(oi.line_total) AS order_amount
  FROM ecom.orders AS o
  JOIN ecom.order_items AS oi
    ON oi.order_id = o.order_id
  WHERE o.order_status IN ('paid', 'shipped', 'delivered')
  GROUP BY
    o.order_id,
    DATE_TRUNC('month', o.order_ts)
)
SELECT
  order_month,
  COUNT(*) AS order_count,
  SUM(order_amount) AS total_sales_amount,
  AVG(order_amount) AS avg_order_amount
FROM order_totals
GROUP BY order_month
ORDER BY order_month;

```

order_month	order_count	total_sales_amount	avg_order_amount
2026-03-01 00:00:00.000 +0900	207	123,172.21	595.0348309179
2026-04-01 00:00:00.000 +0900	1,189	762,757.73	641.5119680404
2026-05-01 00:00:00.000 +0900	1,250	862,701.14	690.160912
2026-06-01 00:00:00.000 +0900	1,373	909,984.31	662.7708011653
2026-07-01 00:00:00.000 +0900	1,044	706,399.18	676.6275670498
2026-08-01 00:00:00.000 +0900	1	177.63	177.63

실행 구조와 공유 버퍼 177은 동일했으며, 갱신된 통계와 캐시 상태에서 집계 시간이 단축됐다.

튜닝 후 JSON

판단 — 월별 전체 리포트는 판매 주문 대부분을 읽어야 하므로 Seq Scan과 HashAggregate가 합리적이다. 이 유형은 인덱스보다 월별 Materialized View를 월별로 재집계하는 방식이 확장성에 유리하다.

CATEGORY PERFORMANCE

최근 90일 카테고리 매출

05

튜닝 전
8.194 ms

튜닝 후
7.834 ms

감소율
4.4%

1위
Women

튜닝 전

orders Seq Scan

```
SELECT
  ct.category_id,
  ct.category_name,
  SUM(oi.line_total) AS category_sales_amount
FROM ecom.orders AS o
JOIN ecom.order_items AS oi ON oi.order_id = o.order_id
JOIN ecom.products AS p ON p.product_id = oi.product_id
JOIN ecom.categories AS ct ON ct.category_id = p.category_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
  AND o.order_ts >= CURRENT_TIMESTAMP - INTERVAL '90 days'
GROUP BY
  ct.category_id,
  ct.category_name
ORDER BY category_sales_amount DESC
LIMIT 10;
```

categories 1 X

SELECT ct.category_id, ct.category_name, SUM(oi.line_total) AS category_sales_amount

	123 category_id	AZ category_name	123 category_sales_amount
1	10	Women	1,301,748.94
2	2	Phones	690,161
3	3	Laptops	86,037.03
4	13	Outdoor	83,116.05
5	9	Men	82,613.94
6	11	Shoes	79,227.14
7	14	Fitness	77,585.55
8	4	Audio	74,342.55
9	7	Cookware	72,650.5
10	6	Appliances	68,561.91

orders → order_items → products → categories를 Hash Join하고 카테고리별 매출을 집계했다.

튜닝 전 JSON

튜닝 후

orders Index Only

```
Q3. 튜닝 후 쿼리
[재출] 튜닝 후 쿼리 및 실행 결과 화면
[비교] EXPLAIN ANALYZE로 튜닝 전후 실행 계획과 실행 시간 비교
[튜닝] 최근 90일 주문을 부분 인덱스로 먼저 줄인 뒤 카테고리 집계

EXPLAIN (ANALYZE, BUFFERS)
SELECT
  ct.category_id,
  ct.category_name,
  SUM(oi.line_total) AS category_sales_amount
FROM ecom.orders AS o
JOIN ecom.order_items AS oi
  ON oi.order_id = o.order_id
JOIN ecom.products AS p
  ON p.product_id = oi.product_id
JOIN ecom.categories AS ct
  ON ct.category_id = p.category_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
  AND o.order_ts >= CURRENT_TIMESTAMP - INTERVAL '90 days'
GROUP BY
  ct.category_id,
  ct.category_name
ORDER BY category_sales_amount DESC
LIMIT 10;
```

categories 1 X

SELECT ct.category_id, ct.category_name, SUM(oi.line_total) AS category_sales_amount

	123 category_id	AZ category_name	123 category_sales_amount
10	10	Women	1,295,632.36
2	2	Phones	690,161
3	3	Laptops	86,037.03
13	13	Outdoor	83,116.05
9	9	Men	82,613.94
11	11	Shoes	79,227.14
14	14	Fitness	77,585.55
4	4	Audio	74,112.99
7	7	Cookware	72,650.5
6	6	Appliances	68,561.91

기간·판매 상태 부분 인덱스로 orders 3,891행을 읽고 동일한 Top 10을 반환했다.

튜닝 후 JSON

결과 — Women 카테고리가 1,301,748.94로 1위였다. 인덱스가 선택됐지만 대상 기간이 넓어 버퍼는 185에서 271로 증가했고 시간 개선도 4.4%로 제한적이었다.

PRODUCT RANKING · TOP 20

제품별 누적매출 순위

06

튜닝 전
25.163 ms

튜닝 후
10.234 ms

감소율
59.3%

속도
2.46×

튜닝 전

CTE 2단계

rank	sales_rank	product_id	product_name	product_sales_amount
1	1	210	Product 224	1,743,812.16
2	2	566	Product 430	616,674.79
3	3	244	Product 565	7,685.62
4	4	120	Product 362	6,102.37
5	5	573	Product 230	5,973.78
6	6	179	Product 13	5,859.43
7	7	536	Product 329	5,096.26
8	8	483	Product 369	5,049.1
9	9	193	Product 244	4,846.58
10	10	542	Product 150	4,679.2
11	11	93	Product 472	4,654.96
12	12	123	Product 563	4,622.27
13	13	325	Product 236	4,521.23
14	14	62	Product 302	4,445.09
15	15	115	Product 272	4,346.93
16	16	524	Product 89	4,341.2
17	17	17	Product 371	4,264.31
18	18	277	Product 15	4,160.89
19	19	86	Product 122	4,092.21
20	20	155	Product 343	4,058.27

제품 매출 CTE와 순위 CTE를 분리했다. Product 224가 1,743,812.16으로 1위였다.

튜닝 전 JSON

튜닝 후

집계+RANK 통합

rank	sales_rank	product_id	product_name	product_sales_amount
1	1	210	Product 224	1,743,812.16
2	2	566	Product 430	616,674.79
3	3	244	Product 565	7,685.62
4	4	120	Product 362	6,102.37
5	5	573	Product 230	5,973.78
6	6	179	Product 13	5,859.43
7	7	536	Product 329	5,096.26
8	8	483	Product 369	5,049.1
9	9	193	Product 244	4,846.58
10	10	542	Product 150	4,679.2
11	11	93	Product 472	4,654.96
12	12	123	Product 563	4,622.27
13	13	325	Product 236	4,521.23
14	14	62	Product 302	4,445.09
15	15	115	Product 272	4,346.93
16	16	524	Product 89	4,341.2
17	17	17	Product 371	4,264.31
18	18	277	Product 15	4,160.89
19	19	86	Product 122	4,092.21
20	20	155	Product 343	4,058.27

제품별 SUM과 RANK 계산을 하나의 CTE에서 수행해 중간 표현을 단순화했다.

튜닝 후 JSON

판단 — 공유 버퍼는 184로 동일하고 주요 노드도 HashAggregate·WindowAgg로 유지됐으나 측정 실행 시간은 59.3% 감소했다. 구조 단순화 효과와 캐시 영향을 함께 고려해야 하며 반복 측정으로 재확인 이 필요하다.

CUSTOMER VALUE

고객별 RFM 분석

07

튜닝 전
12.987 ms튜닝 후
14.895 ms변화
14.7% 증가상위 고객
Customer 25

튜닝 전

주문별 선집계

```

WITH orders_totals AS (
  SELECT
    o.order_id,
    o.customer_id,
    o.order_ts,
    SUM(oj.line_total) AS order_amount
  FROM ecom.orders AS o
  JOIN ecom.order_items AS oj ON o.order_id = oj.order_id
  WHERE o.order_status IN ('paid', 'shipped', 'delivered')
  GROUP BY o.order_id, o.customer_id, o.order_ts
)
SELECT
  cust.customer_id,
  cust.full_name,
  cust.email,
  CURRENT_DATE - MAX(orders_totals.order_ts)::date AS recency,
  COUNT(DISTINCT orders_totals.order_id) AS frequency,
  SUM(orders_totals.order_amount) AS monetary
FROM orders_totals AS ot
JOIN ecom.customers AS cust ON ot.customer_id = cust.customer_id
GROUP BY cust.customer_id, cust.full_name, cust.email
ORDER BY monetary DESC;

```

customer_id	full_name	email	recency	frequency	monetary
25	Customer 25	user25@example.com	0	16	24,034.92
28	Customer 28	user28@example.com	1	16	23,304.15
4	Customer 4	user4@example.com	4	16	19,003.46
2	Customer 2	user2@example.com	3	16	18,927.28
5	Customer 5	user5@example.com	2	16	18,530.68
26	Customer 26	user26@example.com	4	16	17,833.31
6	Customer 6	user6@example.com	6	16	16,734.89
29	Customer 29	user29@example.com	13	16	16,075.16
11	Customer 11	user11@example.com	7	16	15,519.3
18	Customer 18	user18@example.com	1	14	15,381.98
3	Customer 3	user3@example.com	6	16	14,719.9
10	Customer 10	user10@example.com	13	16	14,452.41
20	Customer 20	user20@example.com	2	16	13,788.81
17	Customer 17	user17@example.com	8	16	12,995.34
8	Customer 8	user8@example.com	1	14	12,942.49
19	Customer 19	user19@example.com	1	16	12,847.33
1	Customer 1	user1@example.com	8	16	12,799.47
9	Customer 9	user9@example.com	2	16	12,680.43
30	Customer 30	user30@example.com	3	16	12,369.64

주문을 한 행으로 먼저 줄인 후 고객별 RFM을 계산했다. 최고 Monetary는 24,034.92였다.

튜닝 전 JSON

튜닝 후

직접 고객 집계

```

-- Q5. 튜닝 후 확인
-- [비교] 튜닝 후 처리 및 실행 결과 확인
-- [비교] EXPLAIN ANALYZE로 튜닝 전후 실행 계획과 실행 시간 비교
-- [특징] 주문별 CTE 없이 고객별로 한 번에 집계하고 주문 수는 DISTINCT 처리

EXPLAIN (ANALYZE, BUFFERS)
SELECT
  cust.customer_id,
  cust.full_name,
  cust.email,
  CURRENT_DATE - MAX(o.order_ts)::date AS recency,
  COUNT(DISTINCT o.order_id) AS frequency,
  SUM(oj.line_total) AS monetary
FROM ecom.orders AS o
JOIN ecom.order_items AS oj
ON o.order_id = oj.order_id
JOIN ecom.customers AS cust
ON cust.customer_id = o.customer_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
GROUP BY
  cust.customer_id,
  cust.full_name,
  cust.email
ORDER BY monetary DESC;

```

customer_id	full_name	email	recency	frequency	monetary
25	Customer 25	user25@example.com	0	16	24,034.92
28	Customer 28	user28@example.com	1	16	23,304.15
4	Customer 4	user4@example.com	4	16	19,003.46
2	Customer 2	user2@example.com	3	16	18,927.28
5	Customer 5	user5@example.com	2	16	18,530.68
26	Customer 26	user26@example.com	4	16	17,833.31
6	Customer 6	user6@example.com	6	16	16,734.89
29	Customer 29	user29@example.com	13	16	16,075.16
11	Customer 11	user11@example.com	7	16	15,519.3
18	Customer 18	user18@example.com	1	14	15,381.98
3	Customer 3	user3@example.com	6	16	14,719.9
10	Customer 10	user10@example.com	13	16	14,452.41
20	Customer 20	user20@example.com	2	16	13,788.81
17	Customer 17	user17@example.com	8	16	12,995.34
8	Customer 8	user8@example.com	1	14	12,942.49
19	Customer 19	user19@example.com	1	16	12,847.33
1	Customer 1	user1@example.com	8	16	12,799.47
9	Customer 9	user9@example.com	2	16	12,680.43
30	Customer 30	user30@example.com	3	16	12,369.64
15	Customer 15	user15@example.com	8	16	12,307.67

CTE를 제거하고 COUNT(DISTINCT order_id)로 빈도를 계산했지만 더 많은 상품 행을 GroupAggregate에 전달했다.

튜닝 후 JSON

회귀 분석 — 단일 집계로 단순화했지만 GroupAggregate 입력이 주문 5,064행에서 상품 8,421행 수준으로 늘어 실행 시간이 증가했다. 이 문항은 튜닝 전의 주문별 선집계 구조를 유지하는 것이 더 적절하다.

RETENTION · COHORT

첫 구매 후 30일 내 재구매율

08

튜닝 전
8.567 ms

튜닝 후
3.639 ms

감소율
57.5%

Buffers
10,354 → 146

튜닝 전

상관 EXISTS

```

WITH first_purchases AS (
  SELECT
    o.customer_id,
    MIN(o.order_ts) AS first_order_ts
  FROM ecom.orders AS o
  WHERE o.order_status IN ('paid', 'shipped', 'delivered')
  GROUP BY o.customer_id
  HAVING MIN(o.order_ts) <= CURRENT_TIMESTAMP - INTERVAL '30 days'
),
repurchase_flags AS (
  SELECT
    fp.customer_id,
    fp.first_order_ts,
    EXISTS (
      SELECT 1
      FROM ecom.orders AS next_o
      WHERE next_o.customer_id = fp.customer_id
      AND next_o.order_status IN ('paid', 'shipped', 'delivered')
      AND next_o.order_ts > fp.first_order_ts
      AND next_o.order_ts <= fp.first_order_ts + INTERVAL '30 days'
    ) AS repurchased_within_30d
  FROM first_purchases AS fp
)
SELECT
  COUNT(*) AS first_purchase_customers,
  COUNT(*) FILTER (
    WHERE repurchased_within_30d
  ) AS repurchase_customers,
  ROUND(
    100.0 * COUNT(*) FILTER (WHERE repurchased_within_30d)
    / NULLIF(COUNT(*), 0),
    2
  ) AS repurchased_rate_pct
FROM repurchase_flags;

```

repurchase_flags 1 X

	123 first_purchase_customers	123 repurchase_customers	123 repurchased_rate_pct
1	2,093	957	45.72

2,093명의 성숙 고객마다 다음 주문을 반복 탐색해 957명, 45.72%의 재구매율을 계산했다.

튜닝 전 JSON

튜닝 후

Window 1회 처리

```

-- EXPLAIN (ANALYZE, BUFFERS)
WITH sequenced_orders AS (
  SELECT
    o.customer_id,
    o.order_ts,
    ROW_NUMBER() OVER (
      PARTITION BY o.customer_id
      ORDER BY o.order_ts, o.order_id
    ) AS purchase_no
  FROM ecom.orders AS o
  WHERE o.order_status IN ('paid', 'shipped', 'delivered')
),
first_two_purchases AS (
  SELECT
    customer_id,
    MAX(order_ts) FILTER (WHERE purchase_no = 1) AS first_order_ts,
    MAX(order_ts) FILTER (WHERE purchase_no = 2) AS second_order_ts
  FROM sequenced_orders
  WHERE purchase_no <= 2
  GROUP BY customer_id
),
repurchase_flags AS (
  SELECT
    customer_id,
    second_order_ts <= first_order_ts + INTERVAL '30 days'
    AS repurchased_within_30d
  FROM first_two_purchases
  WHERE first_order_ts <= CURRENT_TIMESTAMP - INTERVAL '30 days'
)
SELECT
  COUNT(*) AS first_purchase_customers,
  COUNT(*) FILTER (
    WHERE repurchased_within_30d
  ) AS repurchase_customers,
  ROUND(
    100.0 * COUNT(*) FILTER (WHERE repurchased_within_30d)
    / NULLIF(COUNT(*), 0),
    2
  ) AS repurchase_rate_pct
FROM repurchase_flags;

```

purchase_flags 1 X

	123 first_purchase_customers	123 repurchase_customers	123 repurchase_rate_pct
	2,093	957	45.72

고객별 구매 순서를 한 번 계산하고 첫 번째·두 번째 구매만 비교했다.

튜닝 후 JSON

핵심 개선 — 기존 계획은 고객당 Index Scan을 두 차례, 총 2,093 loops씩 수행했다. 튜닝 후 부분 커버링 인덱스의 Index Only Scan과 WindowAgg 한 번으로 바뀌어 버퍼가 98.6% 감소했다.

STOCKOUT RISK

재고가 임계치보다 낮은 상품

09

튜닝 전
0.210 ms튜닝 후
0.159 ms감소율
24.3%위험 상품
63개

튜닝 전

Seq Scan

현재 재고가 reorder_point보다 작은 상품을 부족 수량 내림차순으로 정렬했다.

튜닝 전 JSON

튜닝 후

Seq Scan 유지

부분 표현식 인덱스를 추가했지만 inventory 600행 규모에서는 전체 스캔이 더 저렴해 선택되지 않았다.

튜닝 후 JSON

판단 — 실행 시간은 0.051ms 줄었지만 절대 차이가 매우 작고 계획·버퍼(12)가 동일하다. 데이터가 증가하고 위험 상품 비율이 낮아질 때 부분 인덱스의 효과가 커질 수 있다.

REVIEW QUALITY

평점 4.5 이상·리뷰 50개 이상

10

튜닝 전
0.914 ms

튜닝 후
2.062 ms

변화
125.6% 증가

선정 상품
12개

튜닝 전

HashAggregate

```
SELECT
  p.product_id,
  p.product_name,
  ROUND(AVG(rev.rating), 2) AS rating,
  COUNT(*) AS reviews_cnt
FROM ecom.products AS p
JOIN ecom.reviews AS rev ON p.product_id = rev.product_id
GROUP BY p.product_id, p.product_name
HAVING AVG(rev.rating) >= 4.5
AND COUNT(*) >= 50
ORDER BY rating DESC, reviews_cnt DESC;
```

products 1 X

SELECT p.product_id, p.product_name, ROUND(AVG(rev.rating), 2) AS rating, COUNT(*) AS reviews_cnt

Enter a SQL expression to filter results

	123 product_id	A2 product_name	123 rating	123 reviews_cnt
	385	Product 557	4.86	160
	85	Product 112	4.83	160
	394	Product 407	4.81	160
	387	Product 447	4.8	160
	586	Product 250	4.8	160
	57	Product 491	4.79	161
	84	Product 512	4.78	160
	537	Product 319	4.76	160
	417	Product 477	4.75	161
0	341	Product 406	4.75	160
1	238	Product 44	4.74	160
2	432	Product 88	4.73	160

Product 557이 평점 4.86, 리뷰 160개로 가장 높은 평균 평점을 기록했다.

튜닝 전 JSON

튜닝 후

Seq Scan 유지

```
--- Q8. 튜닝 후 쿼리
--- [재출] 튜닝 후 쿼리 및 실행 결과 확인
--- [비교] EXPLAIN ANALYZE로 튜닝 전후 실행 계획과 실행 시간 비교
--- [튜닝] product_id+rating 커버링 인덱스로 제품별 리뷰 집계 보조

--- EXPLAIN (ANALYZE, BUFFERS)
SELECT
  p.product_id,
  p.product_name,
  ROUND(AVG(rev.rating), 2) AS rating,
  COUNT(*) AS reviews_cnt
FROM ecom.reviews AS rev
JOIN ecom.products AS p
  ON p.product_id = rev.product_id
GROUP BY
  p.product_id,
  p.product_name
HAVING AVG(rev.rating) >= 4.5
AND COUNT(*) >= 50
ORDER BY rating DESC, reviews_cnt DESC;
```

products 1 X

SELECT p.product_id, p.product_name, ROUND(AVG(rev.rating), 2) AS rating, COUNT(*) AS reviews_cnt

	product_id	product_name	rating	reviews_cnt
1	385	Product 557	4.86	160
2	85	Product 112	4.83	160
3	394	Product 407	4.81	160
4	387	Product 447	4.8	160
5	586	Product 250	4.8	160
6	57	Product 491	4.79	161
7	84	Product 512	4.78	160
8	537	Product 319	4.76	160
9	417	Product 477	4.75	161
10	341	Product 406	4.75	160
11	238	Product 44	4.74	160
12	432	Product 88	4.73	160

rating 포함 커버링 인덱스를 추가했으나 리뷰 2,055행 대부분을 집계해야 해 옵티마이저가 사용하지 않았다.

튜닝 후 JSON

회귀 분석 — 실행 계획과 버퍼 29는 동일하므로 인덱스 효과가 없고 단일 실행 변동이 크게 나타났다. 이 규모에서는 Seq Scan이 적절하며, 반복 측정 중앙값으로 검증하거나 인덱스 추가를 보여주는 편이 낫다.

COUPON VS NO COUPON

쿠폰 사용 여부별 평균 주문 금액

11

튜닝 전
9.893 ms

튜닝 후
7.877 ms

감소율
20.4%

쿠폰 주문 AOV
2,048.40

튜닝 전

문자열 그룹

```

=====
WITH orders_totals AS (
  SELECT
    o.order_id,
    CASE
      WHEN coupon_code IS NOT NULL THEN 'coupon_used'
      ELSE 'no_coupon'
    END AS coupon_used,
    SUM(oi.line_total) AS order_amount
  FROM ecom.orders AS o
  JOIN ecom.order_items AS oi ON o.order_id = oi.order_id
  WHERE o.order_status IN ('paid', 'shipped', 'delivered')
  GROUP BY o.order_id, o.coupon_code
)
SELECT
  coupon_used,
  AVG(order_amount) AS avg_amount,
  COUNT(*) AS order_count
FROM orders_totals
GROUP BY coupon_used
ORDER BY coupon_used;
=====

```

Results 1 X

TH orders_totals AS (SELECT o.order_id, CASE WHEN co... | Enter a SQL exp

A2 coupon_used	123 avg_amount	123 order_count
coupon_used	2,048.4035844156	1,155
no_coupon	255.637262727	3,909

쿠폰 사용 1,155건의 평균은 2,048.40, 미사용 3,909건은 255.64로 집계됐다.

튜닝 전 JSON

튜닝 후

Boolean 그룹

```

=====
-- Q9. 튜닝 후 쿼리
-- [제출] 튜닝 후 쿼리 및 실행 결과 화면
-- [비교] EXPLAIN ANALYZE로 튜닝 전후 실행 계획과 실행 시간 비교
-- [튜닝] 주문별 금액을 한 번만 집계하고 쿠폰 사용 여부만 상위 집계
=====
-- EXPLAIN (ANALYZE, BUFFERS)
WITH order_totals AS (
  SELECT
    o.order_id,
    o.coupon_code IS NOT NULL AS coupon_used,
    SUM(oi.line_total) AS order_amount
  FROM ecom.orders AS o
  JOIN ecom.order_items AS oi
    ON oi.order_id = o.order_id
  WHERE o.order_status IN ('paid', 'shipped', 'delivered')
  GROUP BY
    o.order_id,
    o.coupon_code
)
SELECT
  CASE
    WHEN coupon_used THEN 'coupon_used'
    ELSE 'no_coupon'
  END AS coupon_group,
  AVG(order_amount) AS avg_order_amount,
  COUNT(*) AS order_count
FROM order_totals
GROUP BY coupon_used
ORDER BY coupon_used DESC;
=====

```

Results 1 X

TH order_totals AS (SELECT o.order_id, o.coupon_code IS NOT NULL. | Enter a SQL

A7 coupon_group	123 avg_order_amount	123 order_count
coupon_used	2,048.4035844156	1,155
no_coupon	255.637262727	3,909

주문별 집계에서는 Boolean으로 분류하고 최종 출력에서만 표시 문자열로 변환했다.

튜닝 후 JSON

해석 — 시드 데이터에서 쿠폰 주문은 더 비싼 상품·더 많은 품목과 수량을 선택하도록 생성됐다. 따라서 쿠폰이 매출을 인과적으로 8배 높였다는 뜻이 아니라, 쿠폰 사용 주문에서 높은 AOV가 관찰됐다는 의미다.

TOP CUSTOMER SEGMENT

상위 1% 고객의 최근 60일 매출

12

튜닝 전

6.072 ms

튜닝 후

4.246 ms

감소율

30.1%

상위 그룹

17명

튜닝 전

orders Seq Scan

```

WITH customer_sales AS (
  SELECT
    o.customer_id,
    SUM(oi.line_total) AS sales_60d
  FROM ecom.orders AS o
  JOIN ecom.order_items AS oi ON o.order_id = oi.order_id
  WHERE o.order_status IN ('paid', 'shipped', 'delivered')
  AND o.order_ts >= CURRENT_TIMESTAMP - INTERVAL '60 days'
  GROUP BY o.customer_id
), ranked_customers AS (
  SELECT
    customer_id,
    sales_60d,
    NTILE(100) OVER (
      ORDER BY sales_60d DESC
    ) AS sales_group
  FROM customer_sales
)
SELECT
  rc.customer_id,
  cust.full_name,
  cust.email,
  rc.sales_60d
FROM ranked_customers AS rc
JOIN ecom.customers AS cust ON cust.customer_id = rc.customer_id
WHERE rc.sales_group = 1
ORDER BY rc.sales_60d DESC;

```

orders(+1) X

Enter a SQL expression to filter results

	125 customer_id	AZ full_name	AZ email	123 sales_60d
1	25	Customer 25	user25@example.com	23,831.11
2	28	Customer 28	user28@example.com	20,426.75
3	4	Customer 4	user4@example.com	19,003.46
4	2	Customer 2	user2@example.com	18,927.28
5	26	Customer 26	user26@example.com	17,833.31
6	6	Customer 6	user6@example.com	16,643.09
7	5	Customer 5	user5@example.com	16,108.38
8	29	Customer 29	user29@example.com	16,075.16
9	18	Customer 18	user18@example.com	15,381.98
10	11	Customer 11	user11@example.com	15,155.86
11	3	Customer 3	user3@example.com	14,719.9
12	10	Customer 10	user10@example.com	14,452.41
13	20	Customer 20	user20@example.com	13,728.01
14	8	Customer 8	user8@example.com	12,942.49
15	17	Customer 17	user17@example.com	12,798.63
16	19	Customer 19	user19@example.com	12,426.49
17	30	Customer 30	user30@example.com	12,369.64

최근 60일 구매 고객을 매출순으로 100개 구간에 나눠 1번 구간 17명을 조회했다.

튜닝 전 JSON

튜닝 후

orders Index Only

```

-- EXPLAIN (ANALYZE, BUFFERS)
WITH customer_sales AS (
  SELECT
    o.customer_id,
    SUM(oi.line_total) AS sales_60d
  FROM ecom.orders AS o
  JOIN ecom.order_items AS oi
  ON o.order_id = oi.order_id
  WHERE o.order_status IN ('paid', 'shipped', 'delivered')
  AND o.order_ts >= CURRENT_TIMESTAMP - INTERVAL '60 days'
  GROUP BY o.customer_id
), ranked_customers AS (
  SELECT
    customer_id,
    sales_60d,
    NTILE(100) OVER (
      ORDER BY sales_60d DESC
    ) AS sales_group
  FROM customer_sales
)
SELECT
  rc.customer_id,
  cust.full_name,
  cust.email,
  rc.sales_60d
FROM ranked_customers AS rc
JOIN ecom.customers AS cust
ON cust.customer_id = rc.customer_id
WHERE rc.sales_group = 1
ORDER BY rc.sales_60d DESC;

```

orders(+1) X

Enter a SQL expression to filter results

	125 customer_id	AZ full_name	AZ email	123 sales_60d
1	25	Customer 25	user25@example.com	23,831.11
2	28	Customer 28	user28@example.com	20,426.75
3	4	Customer 4	user4@example.com	19,003.46
4	2	Customer 2	user2@example.com	18,927.28
5	26	Customer 26	user26@example.com	17,833.31
6	6	Customer 6	user6@example.com	16,643.09
7	5	Customer 5	user5@example.com	16,108.38
8	29	Customer 29	user29@example.com	16,075.16
9	18	Customer 18	user18@example.com	15,381.98
10	11	Customer 11	user11@example.com	15,155.86
11	3	Customer 3	user3@example.com	14,719.9
12	10	Customer 10	user10@example.com	14,452.41
13	20	Customer 20	user20@example.com	13,728.01
14	8	Customer 8	user8@example.com	12,942.49
15	17	Customer 17	user17@example.com	12,798.63
16	19	Customer 19	user19@example.com	12,426.49

부분 인덱스로 최근 주문을 제한한 뒤 동일하게 NTILE(100)을 계산했다. Customer 25가 23,831.11로 1위였다.

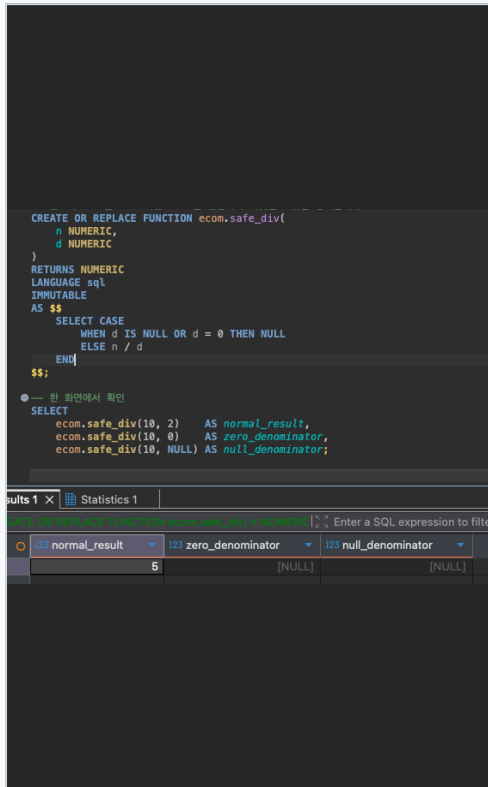
튜닝 후 JSON

개선 — orders 접근이 Seq Scan에서 idx_orders_sales_ts_customer Index Only Scan으로 바뀌어 30.1% 단축됐다. 최종 17명은 customers 기본 키 Index Scan으로 연결됐다.

REUSABLE DATABASE OBJECTS

안전 나눗셈과 일별 GMV

13



ecom.safe_div — 10÷2는 5, 분모 0과 NULL은 오류 대신 NULL을 반환했다.

Materialized View 매번 주문 테이블을 JOIN·SUM하지 않고 일별 GMV를 저장

Daily Report

```

CREATE MATERIALIZED VIEW IF NOT EXISTS ecom.mv_daily_gmv AS
SELECT DATE_TRUNC('day', o.order_ts) AS day,
       SUM(oi.line_total) AS gmv
FROM ecom.orders o
JOIN ecom.order_items oi ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid','shipped','delivered')
GROUP BY DATE_TRUNC('day', o.order_ts);

CREATE UNIQUE INDEX ux_mv_daily_gmv_day
ON ecom.mv_daily_gmv(day);

REFRESH MATERIALIZED VIEW ecom.mv_daily_gmv;

```

갱신 전략 — 데이터 변경 주기에 맞춰 매일 오후 3시 일반 REFRESH를 실행한다. 운영 중 조회 차단을 피해야 하면 고유 인덱스 생성 후 CONCURRENTLY 갱신을 적용한다.

Materialized View 실행 결과
— EXPLAIN (ANALYZE, BUFFERS)

```

SELECT
  day,
  gmv
FROM ecom.mv_daily_gmv
ORDER BY day;

```

v_daily_gmv 1 X

day	gmv
2026-03-26 00:00:00.000 +0900	7,917.03
2026-03-27 00:00:00.000 +0900	22,270.24
2026-03-28 00:00:00.000 +0900	33,082.43
2026-03-29 00:00:00.000 +0900	18,032.49
2026-03-30 00:00:00.000 +0900	27,612.48
2026-03-31 00:00:00.000 +0900	14,257.54
2026-04-01 00:00:00.000 +0900	27,396.82
2026-04-02 00:00:00.000 +0900	10,560.73
2026-04-03 00:00:00.000 +0900	38,938.33
2026-04-04 00:00:00.000 +0900	26,672.98
2026-04-05 00:00:00.000 +0900	15,122.6
2026-04-06 00:00:00.000 +0900	26,948.21
2026-04-07 00:00:00.000 +0900	20,664
2026-04-08 00:00:00.000 +0900	27,104.74
2026-04-09 00:00:00.000 +0900	28,837.72
2026-04-10 00:00:00.000 +0900	25,852.96
2026-04-11 00:00:00.000 +0900	19,380.02
2026-04-12 00:00:00.000 +0900	27,711.46
2026-04-13 00:00:00.000 +0900	28,628.02
2026-04-14 00:00:00.000 +0900	25,985.2
2026-04-15 00:00:00.000 +0900	24,918.83
2026-04-16 00:00:00.000 +0900	31,491.77
2026-04-17 00:00:00.000 +0900	30,502.02
2026-04-18 00:00:00.000 +0900	15,730.1
2026-04-19 00:00:00.000 +0900	34,461.46
2026-04-20 00:00:00.000 +0900	37,905.9
2026-04-21 00:00:00.000 +0900	21,918.31
2026-04-22 00:00:00.000 +0900	17,484.05
2026-04-23 00:00:00.000 +0900	23,309.5
2026-04-24 00:00:00.000 +0900	27,923.6
2026-04-25 00:00:00.000 +0900	22,538.67
2026-04-26 00:00:00.000 +0900	19,049

일별 GMV 전체 조회 결과

최근 30일 범위 조회 결과
— EXPLAIN (ANALYZE, BUFFERS)

```

SELECT day, gmv FROM ecom.mv_daily_gmv WHERE day >= DATE_TRUNC('day', CURRENT_TIMESTAMP) - INTERVAL '30 days'
ORDER BY day;

```

v_daily_gmv 1 X

day	gmv
2026-06-24 00:00:00.000 +0900	26,938.68
2026-06-25 00:00:00.000 +0900	21,020.71
2026-06-26 00:00:00.000 +0900	29,635.03
2026-06-27 00:00:00.000 +0900	22,986.99
2026-06-28 00:00:00.000 +0900	21,227.57
2026-06-29 00:00:00.000 +0900	26,497.41
2026-06-30 00:00:00.000 +0900	35,339.97
2026-07-01 00:00:00.000 +0900	26,657.26
2026-07-02 00:00:00.000 +0900	46,476.24
2026-07-03 00:00:00.000 +0900	33,444.37
2026-07-04 00:00:00.000 +0900	20,363.53
2026-07-05 00:00:00.000 +0900	25,337.64
2026-07-06 00:00:00.000 +0900	37,672.34
2026-07-07 00:00:00.000 +0900	21,117.78
2026-07-08 00:00:00.000 +0900	29,661.33
2026-07-09 00:00:00.000 +0900	34,505.47
2026-07-10 00:00:00.000 +0900	27,894.36
2026-07-11 00:00:00.000 +0900	27,999.13
2026-07-12 00:00:00.000 +0900	46,589
2026-07-13 00:00:00.000 +0900	20,708.07
2026-07-14 00:00:00.000 +0900	19,307.66
2026-07-15 00:00:00.000 +0900	38,460.56
2026-07-16 00:00:00.000 +0900	32,717.33
2026-07-17 00:00:00.000 +0900	25,621.11
2026-07-18 00:00:00.000 +0900	23,215.21
2026-07-19 00:00:00.000 +0900	36,394.72
2026-07-20 00:00:00.000 +0900	37,144.8
2026-07-21 00:00:00.000 +0900	27,469.72
2026-07-22 00:00:00.000 +0900	26,062.35
2026-07-23 00:00:00.000 +0900	23,368.48
2026-07-24 00:00:00.000 +0900	18,078.66
2026-07-30 00:00:00.000 +0900	132.05

최근 30일 범위 조회 결과

BEFORE / AFTER SUMMARY

Q1~Q10 실행 시간 비교

14

문항	튜닝 전	튜닝 후	변화	주요 실행 계획 변화
Q1	3.484 ms	3.502 ms	0.5% 증가	orders Seq Scan → Index Only Scan, 전체 시간은 동일 수준
Q2	12.356 ms	10.908 ms	11.7% 감소	Hash Join·2단계 HashAggregate 유지
Q3	8.194 ms	7.834 ms	4.4% 감소	orders Seq Scan → Index Only Scan
Q4	25.163 ms	10.234 ms	59.3% 감소	집계와 RANK CTE 통합, 주요 노드는 동일
Q5	12.987 ms	14.895 ms	14.7% 증가	주문별 선집계 제거 후 GroupAggregate 입력 증가
Q6	8.567 ms	3.639 ms	57.5% 감소	반복 Index Scan → Index Only Scan + WindowAgg
Q7	0.210 ms	0.159 ms	24.3% 감소	소규모 테이블로 Seq Scan 유지
Q8	0.914 ms	2.062 ms	125.6% 증가	커버링 인덱스 미선택, 동일 계획에서 측정 변동
Q9	9.893 ms	7.877 ms	20.4% 감소	Boolean 그룹화, HashAggregate 유지
Q10	6.072 ms	4.246 ms	30.1% 감소	orders Seq Scan → Index Only Scan
합계	87.840 ms	65.356 ms	25.6% 감소	캡처된 각 1회 실행 시간의 단순 합계

가장 큰 구조적 개선 · Q6

상관 서브쿼리의 반복 접근을 Window 처리로 바꾸면서 shared buffer가 10,354에서 146으로 감소했다.

인덱스 효과 · Q10

판매 상태·기간 부분 인덱스가 최근 60일 조회에 선택돼 30.1% 단축됐다.

회귀 · Q5

CTE 제거가 항상 튜닝은 아니다. 주문별 선집계가 데이터량을 먼저 줄여 더 유리했다.

소규모 데이터 · Q7/Q8

전체 테이블이 작거나 다수 행을 집계하면 옵티마이저가 Seq Scan을 유지하는 것이 정상이다.

PHYSICAL JOIN STRATEGY

세 가지 물리적 JOIN 방식

15

구분	Nested Loop Join	Hash Join	Merge Join
원리	외부 입력의 각 행마다 내부 입력 탐색	작은 입력으로 해시를 만들고 큰 입력을 Probe	정렬된 양쪽 입력을 동시에 순차 탐색
적합한 규모	소량 결과, 높은 선택도	대량 동등 조인	정렬된 대량 입력
인덱스	내부 조인 키 인덱스가 중요	인덱스 없이도 활용 가능	정렬 인덱스가 있으면 Sort 회피
주의점	외부 행이 많으면 내부 탐색 반복 증가	메모리 부족 시 Hash Batch-Spill	입력 미정렬 시 사전 Sort 비용
이번 계획	Q10에서 상위 17명과 customers PK 연결	다수 문항의 orders-order_items 결합	이번 측정에서는 선택되지 않음

Nested Loop

소수 고객을 기본 키로 조회하는 Q10의 최종 JOIN처럼 외부 결과가 작고 내부 인덱스가 있을 때 적합하다.

Hash Join

Q1~Q5, Q8~Q10처럼 주문상품 다수를 동등 조건으로 결합할 때 반복 탐색을 피하고 대량 처리에 유리하다.

Merge Join

양쪽 입력이 조인 키로 정렬됐거나 정렬 비용을 이후 연산에서도 활용할 수 있을 때 경쟁력이 생긴다.

JOIN 선택의 핵심

특정 JOIN 방식을 무조건 강제하는 것이 아니라 통계, 실제 행 수, 인덱스, 메모리와 정렬 상태를 개선해 옵티마이저가 적합한 계획을 고르도록 해야 한다.

DATABASE ENGINE OPTIMIZER

엔진별 실행 계획 특징

16

항목	PostgreSQL	MySQL	Oracle	SQL Server
방식	비용 기반	비용 기반	CBO	비용 기반
실제 계획	EXPLAIN ANALYZE	EXPLAIN ANALYZE / TREE	DBMS_XPLAN DISPLAY_CURSOR	Actual Execution Plan
JOIN	NL·Hash·Merge	NL 계열·Hash	NL·Hash·Sort Merge	NL·Hash Match·Merge
접근 특징	Index/Bitmap Scan, Memoize	ref·range·eq_ref, BKA	Index Range Scan, 병렬·파티션	Index Seek, Key Lookup
계획 관리	통계·설정·확장	힌트·optimizer 설정	SQL Plan Baseline	Query Store Plan Forcing
메모리 주의	work_mem, temp, Hash Batch	내부 temp·hash 비용	TEMP, PGA, 병렬 실행	Memory Grant, TempDB Spill

PostgreSQL

예상 rows와 actual rows, loops, buffers를 함께 보고 ANALYZE 통계와 인덱스를 우선 개선한다.

MySQL

기본 표 형식만으로 Hash Join이 명확하지 않을 수 있어 FORMAT=TREE와 실제 실행 분석을 병행한다.

Oracle

E-Rows와 A-Rows 차이, Buffers·Reads·TempSpc를 확인하고 검증된 계획은 Baseline으로 관리할 수 있다.

SQL Server

Estimated/Actual Rows, Memory Grant, Spill, Seek Predicate를 보고 Query Store로 성능 변화를 추적한다.

공통 원칙 — 엔진마다 노드 이름과 계획 관리 기능은 다르지만 카디널리티 추정, 접근 경로, 반복 횟수, 메모리와 Spill을 함께 본다는 원칙은 동일하다.

CONCLUSION

분석 결과와 개선 방향

17

비즈니스 결과

최근 한 달 GMV는 874,043.78, 30일
재구매율은 45.72%, 쿠폰 사용 주문 AOV는
2,048.40으로 집계됐다.

성능 결과

10개 캡처 실행 시간 합계는 25.6% 감소했으며
Q6의 반복 탐색 제거가 가장 명확한 구조적
개선이었다.

운영 결과

일별 GMV를 Materialized View로 저장하고 오후
3시 갱신 전략과 동시 갱신 조건을 마련했다.

최종 결론

인덱스 생성 자체가 항상 성능 향상을 보장하지 않았다. 선택도가 높은 기간 검색과 반복 접근 제거에서는 효과가 컸지만, 작은 테이블이나 대부분의 행을 집계하는 쿼리에서는 Seq Scan이 합리적이었다. 또한 Q5처럼 CTE를 제거해 SQL을 짧게 만드는 것이 오히려 더 많은 행을 상위 집계에 전달할 수 있음을 확인했다.

Q1~Q10 결과 화면

튜닝 전·후 SQL과 실행 결과 이미지 20개를 문항별로 연결했다.

Q1~Q10 실행 계획

JSON 원본 20개에서 실행 시간, 스캔, 조인, 버퍼를 확인했다.

Q11 함수

정상 나눗셈, 0 분모, NULL 분모의 동작을 검증했다.

Materialized View

생성·인덱스·갱신 스크립트와 전체·30일 조회 결과를 포함했다.

JOIN 전략

Nested Loop, Hash, Merge Join의 원리와 적용 조건을 비교했다.

옵티마이저 조사

PostgreSQL, MySQL, Oracle, SQL Server의 특징을 별도 원문과
요약표로 정리했다.